

GROUP BY & HAVING

Analyse Data by Category and Filter Groups

■ LEARNING OUTCOME

HAVING,

■ Skills You Will Gain

- Use GROUP BY to aggregate data per category
- Combine SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY
- Filter groups using HAVING with aggregate conditions
- Use GROUP BY with multiple columns
- Apply ROLLUP for subtotals and grand totals
- Understand which columns can appear in SELECT with GROUP BY

■ **GROUP BY transforms raw transactional data into business intelligence. Revenue by city, orders by month, customers by segment -- every business report uses GROUP BY. It is the heart of analytics SQL.**

SECTION 1

GROUP BY Fundamentals

The GROUP BY Rule

When you use GROUP BY, every column in SELECT must be either: 1. Listed in the GROUP BY clause, OR 2. Wrapped in an aggregate function (COUNT, SUM, AVG, MIN, MAX) You cannot SELECT a column that is not in GROUP BY and not aggregated -- because SQL would not know WHICH row's value to display for that group.

```
-- COUNT orders per city SELECT city, COUNT(*) AS total_customers, SUM(annual_spend) AS
total_revenue, AVG(annual_spend) AS avg_spend FROM customers GROUP BY city; -- Revenue by month and
year SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mo, COUNT(*) AS orders, SUM(total_amount)
AS revenue, AVG(total_amount) AS avg_order FROM orders GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY yr, mo; -- Multi-column GROUP BY SELECT city, CASE WHEN is_premium THEN 'Premium' ELSE
'Standard' END AS tier, COUNT(*) AS customers, SUM(annual_spend) AS revenue, MIN(annual_spend) AS
min_spend, MAX(annual_spend) AS max_spend FROM customers GROUP BY city, is_premium ORDER BY revenue
DESC; -- WHERE filters rows BEFORE grouping -- Only include delivered orders in the aggregation
SELECT customer_id, COUNT(*) AS order_count, SUM(total_amount) AS total_spent FROM orders WHERE
status = 'delivered' -- filter first, then group GROUP BY customer_id;
```

SECTION 2

HAVING -- Filter Groups

```
-- HAVING filters AFTER grouping (uses aggregates) -- Find cities with more than 100 customers
SELECT city, COUNT(*) AS customer_count, SUM(annual_spend) AS total_revenue FROM customers GROUP BY
city HAVING COUNT(*) > 100 ORDER BY customer_count DESC; -- Find customers who spent more than
50000 total SELECT customer_id, COUNT(*) AS orders, SUM(total_amount) AS total_spent FROM orders
GROUP BY customer_id HAVING SUM(total_amount) > 50000 ORDER BY total_spent DESC; -- WHERE + HAVING
together (filter rows, then filter groups) SELECT category, COUNT(*) AS products, AVG(price) AS
avg_price FROM products WHERE is_available = TRUE -- WHERE: filter rows before grouping GROUP BY
category HAVING AVG(price) > 500 -- HAVING: filter groups after grouping ORDER BY avg_price DESC;
-- Multiple HAVING conditions SELECT customer_id, COUNT(*) AS orders, SUM(total_amount) AS revenue,
AVG(total_amount) AS avg_order FROM orders WHERE YEAR(order_date) = 2024 GROUP BY customer_id
HAVING COUNT(*) >= 5 -- at least 5 orders AND SUM(total_amount) > 20000 -- and spent over 20000
ORDER BY revenue DESC;
```

SECTION 3

ROLLUP and CUBE for Subtotals

```
-- ROLLUP -- adds subtotals and grand total SELECT COALESCE(city, 'ALL CITIES') AS city,
COALESCE(CAST(is_premium AS CHAR), 'ALL TIERS') AS tier, COUNT(*) AS customers, SUM(annual_spend)
AS revenue FROM customers GROUP BY ROLLUP(city, is_premium); -- Output: rows for each city+tier,
subtotal per city, grand total -- CUBE -- all possible subtotals (every combination) SELECT city,
category, SUM(total_amount) AS revenue FROM orders o JOIN products p ON o.product_id = p.product_id
JOIN customers c ON o.customer_id = c.customer_id GROUP BY CUBE(city, category); -- GROUPING SETS
-- specify exactly which groupings you want SELECT city, category, SUM(total_amount) AS revenue
FROM sales_summary GROUP BY GROUPING SETS ( (city, category), -- subtotal by city+category (city),
-- subtotal by city only (category), -- subtotal by category only () -- grand total );
```

■ PRACTICE Q & A

Q1. What is the rule for columns in SELECT when using GROUP BY?

A: Every column in SELECT must be either (1) in the GROUP BY clause, or (2) wrapped in an aggregate function. Violating this causes an error or undefined results.

Q2. What is the difference between WHERE and HAVING?

A: WHERE filters individual rows BEFORE grouping -- cannot use aggregates. HAVING filters groups AFTER GROUP BY -- can use COUNT, SUM, AVG. WHERE is step 3, HAVING is step 5 in execution order.

Q3. Can you use HAVING without GROUP BY?

A: Yes -- HAVING without GROUP BY treats the entire table as one group. Rarely useful but valid: HAVING COUNT() > 100 would check if the whole table has more than 100 rows.*

Q4. What does ROLLUP do?

A: ROLLUP generates subtotals for each level of the grouping hierarchy plus a grand total. GROUP BY ROLLUP(city, category) produces city+category rows, then city-only subtotals, then the grand total.

Q5. How do you find customers who placed more than 3 orders?

A: SELECT customer_id, COUNT() AS orders FROM orders GROUP BY customer_id HAVING COUNT(*) > 3. The COUNT(*) in HAVING counts rows per group after grouping.*

■ GROUP BY & HAVING Cheat Sheet	
GROUP BY col	Group rows by column value
GROUP BY col1, col2	Group by multiple columns
HAVING COUNT(*) > 10	Filter groups by aggregate
WHERE before GROUP BY	Filter rows before grouping
HAVING after GROUP BY	Filter groups after grouping
SUM in HAVING	Aggregate in group filter
COUNT(*) in HAVING	Count per group in filter
COALESCE(col, 'Total')	Label ROLLUP NULL rows
GROUP BY ROLLUP(a,b)	Add subtotals and grand total
ORDER BY SUM(col) DESC	Sort groups by aggregate
GROUP BY EXTRACT(YEAR FROM date)	Group by year
COUNT(DISTINCT col) in HAVING	Unique values per group filter